IPSSI MONTPELLIER · MASTER 2 DATA ENGINEERING · FORMATION APACHE AIRFLOW

TP Jour 1 — Pipeline Énergie & Météo avec Docker

TP Jour 1 — Pipeline Énergie & Météo : Prise en main Airflow via Docker

Formation : Data Engineering — Apache Airflow **École** : IPSSI Montpellier **Niveau** : M2 **Durée estimée** : 2h30
Thème : Corrélation météo / production d'énergie renouvelable en France

---

Contexte professionnel

Vous êtes data engineer chez **RTE** (Réseau de Transport d'Électricité), l'entreprise qui gère le réseau électrique français à haute tension. RTE publie en open data les données de production électrique en temps quasi-réel via la plateforme **éCO2mix**.

Votre équipe Data souhaite mettre en place un **tableau de bord quotidien** qui :

1. Collecte les prévisions météo (ensoleillement, vitesse du vent) pour les principales régions productrices d'énergie renouvelable.
2. Collecte la production effective d'énergie solaire et éolienne par région depuis l'API éCO2mix.
3. Corrèle ces deux sources pour détecter des anomalies : par exemple, un fort ensoleillement qui ne se traduit pas par une production solaire attendue peut indiquer une panne sur le réseau ou un problème de remontée de données.
4. Génère un rapport JSON quotidien exploitable par les équipes métier.

Ce pipeline sera orchestré par **Apache Airflow 2.8**, déployé localement via **Docker Compose** pour ce TP.

Open Data utilisées (sans clé d'API) :

Source	URL	Description
Open-Meteo	`https://api.open-meteo.com/v1/forecast`	Prévisions météo horaires gratuites
éCO2mix (OpenDataSoft)	`https://odre.opendatasoft.com/api/explore/v2.1/catalog/datasets/eco2mix-regional-cons-def/records`	Production électrique par région

Prérequis

- Docker Desktop installé et démarré
- Docker Compose v2 (`docker compose` sans tiret)
- Python 3.9+ (pour tester les appels API en local si besoin)
- Environ 4 Go de RAM disponibles pour Docker
- Port 8080 libre sur votre machine

Partie 1 — Setup de l'environnement Docker

1.1 Structure des répertoires

Créez l'arborescence suivante dans votre répertoire de travail :

📐 **Schéma d'architecture**

```
airflow-energie/
    ├── docker-compose.yaml
    ├── .env
    ├── dags/
    │   └── energie_meteo_dag.py
    ├── logs/
    └── plugins/
```

$ Bash

```bash
mkdir -p airflow-energie/{dags,logs,plugins}
    cd airflow-energie
```

1.2 Fichier `.env`

```bash
AIRFLOW_UID=50000
    AIRFLOW_GID=0
```

1.3 Fichier `docker-compose.yaml`

Ce fichier démarre Airflow 2.8 avec le **LocalExecutor** (adapté au développement local) et une base **PostgreSQL** pour les métadonnées.

⚙ YAML

```yaml
version: '3.8'

    x-airflow-common: &airflow-common
      image: apache/airflow:2.8.0
      environment:
        AIRFLOW__CORE__EXECUTOR: LocalExecutor
        AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/ai
        AIRFLOW__CORE__FERNET_KEY: ''
        AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
        AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
        AIRFLOW__API__AUTH_BACKENDS: 'airflow.api.auth.backend.basic_auth,airflow.api.auth.bac
        AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: 'true'
        _PIP_ADDITIONAL_REQUIREMENTS: 'requests'
      volumes:
        - ./dags:/opt/airflow/dags
        - ./logs:/opt/airflow/logs
        - ./plugins:/opt/airflow/plugins
      user: "${AIRFLOW_UID:-50000}:0"
      depends_on:
        postgres:
          condition: service_healthy

    services:
      postgres:
        image: postgres:13
        environment:
          POSTGRES_USER: airflow
          POSTGRES_PASSWORD: airflow
          POSTGRES_DB: airflow
        volumes:
          - postgres-db-volume:/var/lib/postgresql/data
        healthcheck:
          test: ["CMD", "pg_isready", "-U", "airflow"]
          interval: 10s
          retries: 5
          start_period: 5s
        restart: always

      airflow-webserver:
        <<: *airflow-common
        command: webserver
        ports:
          - "8080:8080"
        healthcheck:
          test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
          interval: 30s
          timeout: 10s
          retries: 5
          start_period: 30s
        restart: always

      airflow-scheduler:
        <<: *airflow-common
        command: scheduler
```

```
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 10
      start_period: 30s
    restart: always

  airflow-init:
    <<: *airflow-common
    entrypoint: /bin/bash
    command:
      - -c
      - |
        airflow db init &&
        airflow users create \
          --username admin \
          --firstname Admin \
          --lastname User \
          --role Admin \
          --email admin@example.com \
          --password admin
    environment:
      <<: *airflow-common
      _AIRFLOW_DB_UPGRADE: 'true'
      _AIRFLOW_WWW_USER_CREATE: 'true'
      _AIRFLOW_WWW_USER_USERNAME: admin
      _AIRFLOW_WWW_USER_PASSWORD: admin

volumes:
  postgres-db-volume:
```

1.4 Démarrage

Schéma d'architecture

```
# Initialiser la base de données Airflow (à faire une seule fois)
    docker compose up airflow-init

    # Démarrer tous les services en arrière-plan
    docker compose up -d

    # Vérifier que les conteneurs sont bien démarrés
    docker compose ps

    # Suivre les logs du scheduler
    docker compose logs -f airflow-scheduler
```

Accédez à l'interface web : **http://localhost:8080** Identifiants : `admin` / `admin`

Partie 2 — Présentation des APIs

2.1 Open-Meteo

Open-Meteo est une API météo gratuite et open source, sans clé d'API requise.

Exemple de requête pour Paris (Île-de-France) :

```
GET https://api.open-meteo.com/v1/forecast
  ?latitude=48.8566
  &longitude=2.3522
  &daily=sunshine_duration,wind_speed_10m_max
  &timezone=Europe/Paris
  &forecast_days=1
```

Paramètres clés :

Paramètre	Description
`latitude`, `longitude`	Coordonnées géographiques
`daily=sunshine_duration`	Durée d'ensoleillement en secondes/jour
`daily=wind_speed_10m_max`	Vitesse max du vent à 10m (km/h)
`timezone`	Timezone pour l'interprétation des dates
`forecast_days=1`	Seulement aujourd'hui

Exemple de réponse JSON :

```JSON
{ } JSON

{
      "latitude": 48.8566,
      "longitude": 2.3522,
      "timezone": "Europe/Paris",
      "daily": {
        "time": ["2024-03-15"],
        "sunshine_duration": [28340.5],
        "wind_speed_10m_max": [18.7]
      }
    }
```

Note : `sunshine_duration` est en secondes. Divisez par 3600 pour obtenir des heures.

2.2 API éCO2mix (OpenDataSoft)

Cette API expose les données de production électrique consolidées par région, publiées par ODRÉ (Observatoire des Données des Énergies).

Exemple de requête :

```
GET https://odre.opendatasoft.com/api/explore/v2.1/catalog/datasets/eco2mix-regional-cons-def/
  ?limit=10
  &timezone=Europe%2FParis
```

Exemple de réponse JSON (extrait d'un enregistrement) :

```JSON
{ } JSON

{
      "total_count": 1548,
      "results": [
        {
          "libelle_region": "Île-de-France",
          "date": "2024-03-15",
          "heure": "12:00",
          "solaire": 342.0,
          "eolien": 87.0,
          "consommation": 8421.0,
          "thermique": 0.0,
          "nucleaire": 0.0,
          "hydraulique": 12.0
        }
      ]
    }
```

Champs importants :

Champ	Unité	Description
`libelle_region`	—	Nom de la région
`solaire`	MW	Production solaire
`eolien`	MW	Production éolienne
`consommation`	MW	Consommation totale
`date` + `heure`	—	Horodatage de la mesure

Partie 3 — Architecture du DAG

Le DAG `energie_meteo_dag.py` comporte **5 tâches** s'exécutant quotidiennement à 6h00 (heure de Paris) :

📐 Schéma d'architecture

```
┌──────────────────────────────────────────────────────────────┐
│                    energie_meteo_dag                         │
│              schedule: 0 6 * * *  (chaque jour à 6h)         │
└──────────────────────────────────────────────────────────────┘

            ┌──────────────────────┐
            │    verifier_apis     │  Teste la disponibilité des deux APIs
            └──────────┬───────────┘
                       │
            ┌──────────▼──────────────────────────────┐
            │                                         │
     ┌──────▼──────────────────┐      ┌──────────────▼──────────┐
     │  collecter_meteo_regions │      │ collecter_production_electrique  │
     │  (5 régions Open-Meteo)  │      │  (éCO2mix par région)            │
     └──────────┬──────────────┘      └──────────────┬──────────┘
            │                                         │
            └──────────────────────────────────────────┘
                                   │
                        ┌──────────▼──────────┐
                        │  analyser_correlation│  Corrèle météo et production
                        └──────────┬──────────┘
                                   │
                        ┌──────────▼──────────┐
                        │ generer_rapport_     │  Rapport JSON + tableau CLI
                        │     energie         │
                        └──────────────────────┘
```

Pattern de dépendances :

```python
t1 >> [t2, t3] >> t4 >> t5
```

Partie 4 — Travail demandé

Étape 1 — Démarrer l'environnement Docker et accéder à l'UI

1. Suivez les instructions de la Partie 1 pour créer la structure de fichiers.
2. Lancez `docker compose up -d` depuis le répertoire `airflow-energie/`.
3. Attendez que tous les conteneurs soient en état `healthy` (`docker compose ps`).
4. Ouvrez **http://localhost:8080** et connectez-vous avec `admin` / `admin`.
5. Explorez l'interface : menu DAGs, Admin > Connections, Admin > Variables.

Vérification : Vous devez voir l'interface Airflow sans DAG d'exemple (puisque `LOAD_EXAMPLES=false`).

Étape 2 — Créer le fichier DAG

Créez le fichier `airflow-energie/dags/energie_meteo_dag.py` avec le squelette suivant :

📐 **Schéma d'architecture**

```
from airflow import DAG
    from airflow.operators.python import PythonOperator
    from datetime import datetime, timedelta
    import pendulum
    import requests
    import logging
    import json
    from datetime import date

    # Timezone Paris pour RTE
    local_tz = pendulum.timezone("Europe/Paris")

    # Coordonnées des 5 régions cibles
    REGIONS = {
        "Île-de-France":        {"lat": 48.8566, "lon": 2.3522},
        "Occitanie":            {"lat": 43.6047, "lon": 1.4442},
        "Nouvelle-Aquitaine":   {"lat": 44.8378, "lon": -0.5792},
        "Auvergne-Rhône-Alpes": {"lat": 45.7640, "lon": 4.8357},
        "Hauts-de-France":      {"lat": 50.6292, "lon": 3.0573},
    }

    default_args = {
        "owner": "rte-data-team",
        "depends_on_past": False,
        "email_on_failure": False,
        "retries": 2,
        "retry_delay": timedelta(minutes=5),
    }

    # --- Fonctions à implémenter (étapes 3 à 7) ---

    def verifier_apis(**context):
        pass  # Étape 3

    def collecter_meteo_regions(**context):
        pass  # Étape 4

    def collecter_production_electrique(**context):
        pass  # Étape 5

    def analyser_correlation(**context):
        pass  # Étape 6

    def generer_rapport_energie(**context):
        pass  # Étape 7


    # --- Définition du DAG ---

    with DAG(
        dag_id="energie_meteo_dag",
        default_args=default_args,
        description="Corrélation météo / production énergétique — RTE",
```

```
        schedule="0 6 * * *",
        start_date=datetime(2024, 1, 1, tzinfo=local_tz),
        catchup=False,
        tags=["rte", "energie", "meteo", "open-data"],
    ) as dag:

        t1 = PythonOperator(
            task_id="verifier_apis",
            python_callable=verifier_apis,
        )

        t2 = PythonOperator(
            task_id="collecter_meteo_regions",
            python_callable=collecter_meteo_regions,
        )

        t3 = PythonOperator(
            task_id="collecter_production_electrique",
            python_callable=collecter_production_electrique,
        )

        t4 = PythonOperator(
            task_id="analyser_correlation",
            python_callable=analyser_correlation,
        )

        t5 = PythonOperator(
            task_id="generer_rapport_energie",
            python_callable=generer_rapport_energie,
        )

        t1 >> [t2, t3] >> t4 >> t5
```

Note importante sur XCom : Un `return` dans une `PythonOperator` pousse automatiquement la valeur retournée dans XCom. La tâche `t4` récupère ces valeurs via `ti.xcom_pull(task_ids="nom_de_la_tache")`.

Une fois le fichier créé, Airflow le détectera automatiquement dans les 30 secondes (polling du scheduler). Vérifiez qu'il apparaît dans l'UI sans erreur d'import (icône rouge = erreur de syntaxe Python).

Étape 3 — Implémenter `verifier_apis()`

Cette fonction vérifie que les deux APIs sont joignables avant de lancer les collectes. Si une API est indisponible, le pipeline doit s'arrêter immédiatement plutôt que de continuer avec des données incomplètes.

Schéma d'architecture

```
def verifier_apis(**context):
        """
        Vérifie la disponibilité des APIs Open-Meteo et éCO2mix.
        Lève une exception si une API est indisponible pour bloquer le pipeline.
        """
        apis = {
            "Open-Meteo": (
                "https://api.open-meteo.com/v1/forecast"
                "?latitude=48.8566&longitude=2.3522"
                "&daily=sunshine_duration&timezone=Europe/Paris&forecast_days=1"
            ),
            "éCO2mix": (
                "https://odre.opendatasoft.com/api/explore/v2.1/catalog/datasets"
                "/eco2mix-regional-cons-def/records?limit=1&timezone=Europe%2FParis"
            ),
        }

        for nom, url in apis.items():
            # À compléter :
            # 1. Effectuer un GET sur l'URL avec un timeout de 10 secondes
            # 2. Vérifier que le status code est 200
            # 3. Logger le résultat avec logging.info()
            # 4. En cas d'échec (exception réseau ou status != 200),
            #    lever une ValueError avec un message explicite incluant le nom de l'API
            pass

        logging.info("Toutes les APIs sont disponibles. Pipeline autorisé à continuer.")
```

Travail demandé : Complétez la fonction. Testez-la en déclenchant manuellement le DAG depuis l'UI (bouton "Trigger DAG" en haut à droite de la page du DAG).

Étape 4 — Implémenter `collecter_meteo_regions()`

Schéma d'architecture

```
def collecter_meteo_regions(**context):
        """
        Collecte pour chaque région : durée d'ensoleillement (h) et vitesse max du vent (km/h
        Retourne un dictionnaire {region: {ensoleillement_h: float, vent_kmh: float}}.
        Ce dictionnaire sera automatiquement stocké dans XCom via le return.
        """
        BASE_URL = "https://api.open-meteo.com/v1/forecast"
        resultats = {}

        for region, coords in REGIONS.items():
            params = {
                "latitude": coords["lat"],
                "longitude": coords["lon"],
                "daily": "sunshine_duration,wind_speed_10m_max",
                "timezone": "Europe/Paris",
                "forecast_days": 1,
            }
            # À compléter :
            # 1. Appeler l'API avec requests.get(BASE_URL, params=params, timeout=15)
            # 2. Appeler .raise_for_status() pour déclencher une exception HTTP si besoin
            # 3. Parser le JSON retourné avec .json()
            # 4. Extraire daily["sunshine_duration"][0] → diviser par 3600 pour les heures
            # 5. Extraire daily["wind_speed_10m_max"][0]
            # 6. Stocker dans resultats[region] = {"ensoleillement_h": ..., "vent_kmh": ...}
            # 7. Logger les valeurs avec logging.info()
            pass

        return resultats  # Sera poussé dans XCom automatiquement
```

Résultat attendu (structure de retour) :

Schéma d'architecture

```
{
        "Île-de-France":        {"ensoleillement_h": 7.87, "vent_kmh": 18.7},
        "Occitanie":            {"ensoleillement_h": 9.12, "vent_kmh": 12.3},
        "Nouvelle-Aquitaine":   {"ensoleillement_h": 8.45, "vent_kmh": 24.1},
        "Auvergne-Rhône-Alpes": {"ensoleillement_h": 6.33, "vent_kmh": 31.8},
        "Hauts-de-France":      {"ensoleillement_h": 3.21, "vent_kmh": 42.5},
    }
```

Vérifiez la valeur XCom dans l'UI : cliquez sur l'exécution du DAG > tâche `collecter_meteo_regions` > onglet **XCom**.

Étape 5 — Implémenter `collecter_production_electrique()`

📐 **Schéma d'architecture**

```
def collecter_production_electrique(**context):
        """
        Collecte depuis éCO2mix la production solaire et éolienne par région.
        Agrège les valeurs horaires pour obtenir la moyenne journalière en MW.
        Retourne un dictionnaire {region: {solaire_mw: float, eolien_mw: float}}.
        """
        BASE_URL = (
            "https://odre.opendatasoft.com/api/explore/v2.1/catalog/datasets"
            "/eco2mix-regional-cons-def/records"
        )
        params = {
            "limit": 100,
            "timezone": "Europe/Paris",
        }

        # À compléter :
        # 1. Appeler l'API et récupérer response.json()["results"]
        # 2. Initialiser un dict intermédiaire pour accumuler les valeurs par région :
        #    accumulation = {region: {"solaire": [], "eolien": []} for region in REGIONS}
        # 3. Pour chaque enregistrement :
        #    - Récupérer enregistrement["libelle_region"]
        #    - Si la région est dans REGIONS, ajouter les valeurs solaire et eolien
        #      (remplacer None par 0.0 avec : valeur = enregistrement.get("solaire") or 0.0)
        # 4. Calculer la moyenne par région (sum / len de la liste)
        # 5. Construire et retourner le dict final {region: {solaire_mw, eolien_mw}}

        production = {}
        # ... votre code ici ...
        return production
```

Attention : L'API éCO2mix peut retourner `null` pour certaines valeurs (absence de production sur une période). Gérez systématiquement ce cas avec `valeur or 0.0` ou `valeur if valeur is not None else 0.0`.

Étape 6 — Implémenter `analyser_correlation()`

Cette tâche est le cœur analytique du pipeline. Elle récupère les données des deux tâches précédentes via XCom et applique des règles métier définies par les équipes de RTE.

Schéma d'architecture

```python
def analyser_correlation(**context):
        """
        Corrèle les données météo et les données de production.

        Règles métier RTE :
          - Si ensoleillement > 6h  ET production solaire <= 1000 MW  → ALERTE solaire
          - Si vent > 30 km/h       ET production éolienne <= 2000 MW → ALERTE éolien

        Retourne un dict d'alertes par région, sera stocké dans XCom.
        """
        ti = context["ti"]

        # Récupération des XCom des tâches précédentes
        donnees_meteo = ti.xcom_pull(task_ids="collecter_meteo_regions")
        donnees_production = ti.xcom_pull(task_ids="collecter_production_electrique")

        alertes = {}

        for region in REGIONS:
            meteo = donnees_meteo.get(region, {})
            production = donnees_production.get(region, {})

            alertes_region = []

            ensoleillement = meteo.get("ensoleillement_h", 0)
            vent = meteo.get("vent_kmh", 0)
            solaire = production.get("solaire_mw", 0)
            eolien = production.get("eolien_mw", 0)

            # À compléter — Règle 1 :
            # Si ensoleillement > 6 ET solaire <= 1000 :
            #   alertes_region.append(f"ALERTE SOLAIRE : {ensoleillement:.1f}h de soleil "
            #                         f"mais seulement {solaire:.0f} MW produits")

            # À compléter — Règle 2 :
            # Si vent > 30 ET eolien <= 2000 :
            #   alertes_region.append(f"ALERTE ÉOLIEN : vent à {vent:.1f} km/h "
            #                         f"mais seulement {eolien:.0f} MW produits")

            # Bonus — Règle 3 :
            # Si solaire > 0 ET ensoleillement == 0 → anomalie de données
            # (production mesurée sans ensoleillement enregistré → suspect)

            alertes[region] = {
                "alertes": alertes_region,
                "ensoleillement_h": ensoleillement,
                "vent_kmh": vent,
                "solaire_mw": solaire,
                "eolien_mw": eolien,
                "statut": "ALERTE" if alertes_region else "OK",
            }

        nb_alertes = sum(1 for r in alertes.values() if r["statut"] == "ALERTE")
```

```
        logging.warning(f"{nb_alertes} région(s) en alerte sur {len(REGIONS)} analysées.")

        return alertes
```

Étape 7 — Implémenter `generer_rapport_energie()`

Schéma d'architecture

```
def generer_rapport_energie(**context):
        """
        Génère un rapport JSON et affiche un tableau comparatif dans les logs Airflow.
        Sauvegarde le rapport dans /tmp/rapport_energie_<YYYY-MM-DD>.json.
        Retourne le chemin du fichier généré (stocké dans XCom).
        """
        ti = context["ti"]
        analyse = ti.xcom_pull(task_ids="analyser_correlation")
        today = date.today().isoformat()

        # --- Affichage tableau dans les logs Airflow ---
        print("\n" + "=" * 80)
        print(f"  RAPPORT ENERGIE & METEO — RTE — {today}")
        print("=" * 80)
        print(
            f"{'Region':<25} {'Soleil (h)':>10} {'Vent (km/h)':>12} "
            f"{'Solaire (MW)':>13} {'Eolien (MW)':>12} {'Statut':>8}"
        )
        print("-" * 80)

        for region, data in analyse.items():
            # À compléter :
            # Afficher chaque ligne du tableau avec les valeurs formatées.
            # Exemple de format :
            # print(f"{region:<25} {data['ensoleillement_h']:>10.1f} {data['vent_kmh']:>12.1f
            #       f"{data['solaire_mw']:>13.0f} {data['eolien_mw']:>12.0f} {data['statut']:
            pass

        print("=" * 80 + "\n")

        # --- Construction du rapport JSON ---
        rapport = {
            "date": today,
            "source": "RTE eCO2mix + Open-Meteo",
            "pipeline": "energie_meteo_dag",
            "regions": analyse,
            "resume": {
                "nb_regions_analysees": len(analyse),
                "nb_alertes": sum(1 for r in analyse.values() if r["statut"] == "ALERTE"),
                "regions_en_alerte": [
                    r for r, d in analyse.items() if d["statut"] == "ALERTE"
                ],
            },
        }

        chemin = f"/tmp/rapport_energie_{today}.json"
        with open(chemin, "w", encoding="utf-8") as f:
            json.dump(rapport, f, ensure_ascii=False, indent=2)
```

```
        logging.info(f"Rapport sauvegarde : {chemin}")
        return chemin
```

Vérification : Une fois le DAG exécuté avec succès, lisez le rapport depuis le conteneur :

```bash
$ Bash

docker compose exec airflow-scheduler cat /tmp/rapport_energie_$(date +%Y-%m-%d).json
```

Partie 5 — Questions de réflexion

Répondez par écrit dans un fichier `REPONSES.md` à rendre avec le TP.

Q1 — Docker Executor Le `docker-compose.yaml` utilise `LocalExecutor`. Expliquez la différence entre `LocalExecutor`, `CeleryExecutor` et `KubernetesExecutor`. Dans quel contexte de production RTE devrait-il utiliser chacun ? Quelles sont les implications en termes de scalabilité et de ressources ?

Q2 — Volumes Docker et persistance des DAGs Le volume `./dags:/opt/airflow/dags` permet de modifier les DAGs sans redémarrer le conteneur. Expliquez le mécanisme sous-jacent (bind mount vs volume nommé). Que se passerait-il si on supprimait ce mapping ? Quel serait l'impact en production sur un cluster Airflow multi-nœuds (plusieurs workers) ?

Q3 — Idempotence et `catchup` Le DAG a `catchup=False`. Expliquez ce que ferait Airflow si `catchup=True` et que le DAG est activé aujourd'hui avec un `start_date` au 1er janvier 2024. Qu'est-ce que l'idempotence d'un DAG et pourquoi est-ce critique pour un pipeline de données énergétiques ? Comment rendre les fonctions `collecter_*` idempotentes ?

Q4 — Timezone et données temps-réel L'API éCO2mix retourne des données horodatées. Pourquoi le paramètre `timezone=Europe/Paris` est-il essentiel dans le contexte RTE ? Que peut-il se passer lors du passage à l'heure d'été (dernier dimanche de mars) si la timezone n'est pas correctement gérée dans le scheduler Airflow et dans les requêtes API ? Donnez un exemple concret de données corrompues ou manquantes.

Livrables

À déposer sur le portail de la formation avant la fin de séance :

📐 **Schéma d'architecture**

```
airflow-energie/
    ├── docker-compose.yaml         (fourni, peut être modifié si justifié)
    ├── .env
    ├── dags/
    │   └── energie_meteo_dag.py    DAG complet et fonctionnel
    └── REPONSES.md                  Réponses aux 4 questions avec captures d'écran
```

Captures d'écran à inclure dans `REPONSES.md` :

1. Interface Airflow UI avec le DAG `energie_meteo_dag` visible et en état `success`
2. Vue Graph du DAG montrant les 5 tâches et leurs dépendances
3. Logs d'une exécution réussie de `generer_rapport_energie` avec le tableau affiché
4. Onglet XCom de la tâche `analyser_correlation` montrant le dictionnaire d'alertes
5. Contenu du fichier JSON généré (via `docker compose exec ... cat /tmp/rapport_energie_*.json`)

TP concu pour la formation Data Engineering — IPSSI Montpellier — Apache Airflow 2.8

Exercices Supplémentaires

Ces 3 exercices sont à réaliser après avoir complété le TP principal. Ils approfondissent des concepts clés d'Airflow utilisés en production.

Exercice 1 — SLA et alertes de délai (⭐⭐)

Contexte : En production chez RTE, le rapport météo-énergie doit impérativement être disponible avant 8h pour les équipes d'exploitation. Si le pipeline tarde, une alerte doit être déclenchée.

Objectif : Configurer des SLA (Service Level Agreements) sur les tâches critiques du DAG.

CONCEPT SLA DANS AIRFLOW

Un SLA définit le délai maximum acceptable entre le `start_date` et la fin d'une tâche. Si ce délai est dépassé, Airflow génère un **SLA miss** (visible dans `Browse > SLA Misses`) et peut envoyer un email.

📐 Schéma d'architecture

```
# Exemple de configuration SLA
    from datetime import timedelta

    @task(sla=timedelta(minutes=30))  # La tâche doit finir dans les 30 min après le start_da
    def ma_tache_critique() -> None:
        ...

    # Callback SLA — appelé quand un SLA miss se produit
    def sla_miss_callback(dag, task_list, blocking_task_list, slas, blocking_tis):
        log.warning(
            "[SLA MISS] Les tâches suivantes ont dépassé leur SLA : %s",
            [t.task_id for t in blocking_task_list]
        )
```

TRAVAIL DEMANDÉ

1. Ajoutez un **SLA de 45 minutes** sur la tâche `generer_rapport_energie` (elle doit finir dans les 45 min suivant le `start_date` du DAG).
2. Ajoutez un **SLA de 90 minutes** sur le DAG entier via `default_args["sla"]`.
3. Implémentez la fonction `sla_miss_callback(dag, task_list, blocking_task_list, slas, blocking_tis)` qui :
   - Loggue les tâches en retard avec leurs délais
   - Simule l'envoi d'une notification (log structuré `[ALERTE SLA]`)
4. Injectez le callback dans le DAG :

```python
@dag(
        ...
        sla_miss_callback=sla_miss_callback,
    )
```

5. **Test** : Réduisez le SLA à 1 seconde, déclenchez le DAG, vérifiez dans `Browse > SLA Misses`.

Questions de réflexion : - Quelle est la différence entre `sla` et `execution_timeout` ? - Pourquoi un SLA miss n'arrête-t-il pas la tâche ?

Exercice 2 — Dynamic Task Mapping avec `.expand()` (⭐⭐⭐)

Contexte : Votre pipeline traite 5 régions fixes. Demain, RTE ajoute 8 nouvelles régions. Modifier le code à chaque fois n'est pas scalable.

Objectif : Remplacer la boucle statique par du **Dynamic Task Mapping** Airflow 2.3+.

CONCEPT DYNAMIC TASK MAPPING

Au lieu de créer des tâches en boucle avec `.override(task_id=...)`, le Dynamic Task Mapping génère automatiquement des instances de tâches à partir d'une liste :

Schéma d'architecture

```
@task()
    def traiter_element(element: dict) -> dict:
        # Cette tâche sera instanciée N fois automatiquement
        return {"result": element["nom"]}

    # .expand() crée une instance par élément de la liste
    resultats = traiter_element.expand(
        element=[{"nom": "Paris"}, {"nom": "Lyon"}, {"nom": "Marseille"}]
    )
```

TRAVAIL DEMANDÉ

1. Créez une tâche `charger_config_regions()` qui retourne la liste des régions depuis une **Variable Airflow** `regions_energie` (JSON) :

   Python

   ```
   # Valeur de la variable dans Admin > Variables :
       # [{"nom": "Île-de-France", "lat": 48.8566, "lon": 2.3522, "seuil": 800}, ...]
   ```

2. Remplacez la tâche `extraire_meteo()` par une version compatible Dynamic Mapping :

   Schéma d'architecture

   ```
   @task()
       def extraire_meteo_region(region: dict) -> dict:
           # Reçoit UN seul dict région
           ...

       # Dans le DAG :
       regions = charger_config_regions()
       meteos  = extraire_meteo_region.expand(region=regions)
   ```

3. Adaptez `analyser_correlation()` pour recevoir la liste de résultats du mapping.

4. **Vérification** : Dans l'UI Airflow, la tâche `extraire_meteo_region` doit apparaître avec des sous-instances `[0]`, `[1]`, `[2]`...

5. **Bonus** : Ajoutez une variable `regions_exclues` (liste de noms) et filtrez dynamiquement avant le mapping.

Questions de réflexion : - Quand préférer `.expand()` plutôt que la boucle `.override()` ? - Quelle est la limite de taille des données passées via XCom dans `.expand()` ?

Exercice 3 — Débogage d'un DAG cassé (⭐⭐)

Contexte : Un collègue vous a remis ce DAG qui "ne marche pas". Votre mission : identifier et corriger les 5 erreurs.

LE DAG DÉFECTUEUX

Créez le fichier `dags/dag_broken.py` avec ce contenu et corrigez-le :

📐 **Schéma d'architecture**

```
from __future__ import annotations
    from datetime import datetime, timedelta
    from airflow.decorators import dag, task
    from airflow.models.baseoperator import chain

    @dag(
        dag_id="dag_broken",
        start_date=datetime.now(),        # ERREUR 1
        schedule="@daily",
        catchup=True,                     # ERREUR 2
        default_args={
            "retries": 0,                 # ERREUR 3
            "owner": "stagiaire",
        }
    )
    def dag_broken() -> None:

        @task()
        def extraire() -> dict:
            import requests
            resp = requests.get("https://api.open-meteo.com/v1/forecast"
                                "?latitude=48.8566&longitude=2.3522&current_weather=true")
            return resp.json()["current_weather"]

        @task()
        def transformer(data):             # ERREUR 4 — type hint manquant
            return {
                "temperature": data["temperature"],
                "vent":        data["windspeed"],
                "statut":      "OK" if data["windspeed"] < 50 else "ALERTE",
            }

        @task()
        def charger(resultat: dict) -> None:
            print(f"Résultat : {resultat}")  # ERREUR 5

        data     = extraire()
        resultat = transformer(data)
        charger(resultat)
        # Dépendance manquante : chain non utilisé → Pylance warning

    dag_broken()   # Pas d'assignation à dag_instance
```

TRAVAIL DEMANDÉ

1. Identifiez et expliquez chacune des 5 erreurs.
2. Corrigez le DAG.
3. Ajoutez la dépendance `chain()` manquante.
4. Déclenchez le DAG corrigé et vérifiez son exécution.

Tableau de réponses (à remplir dans `REPONSES.md`) :

#	Ligne	Erreur	Impact	Correction
1	8	`datetime.now()` comme `start_date`		
2	10	`catchup=True`		
3	12	`retries: 0`		
4	27	Type hint manquant		
5	34	`print()` utilisé		

Exercices supplémentaires — Jour 1 — Formation Airflow IPSSI Montpellier